



Jun 21, 2026

Testing Report

Key Insights From TestSprite's Autonomous
AI Testing Agent

Report Contents

1	Executive Summary	03
2	Frontend UI Test Results	03

41

Test Runs

97.5%

Pass Rate

2

Issues

3h

Saved By TestSprite

Table of Contents

3 Executive Summary

3 High-Level Overview

3 Key Findings

3 Frontend UI Test Results

3 Test Coverage Summary

3 Test Execution Summary

6 Test Execution Breakdown

Executive Summary

1 High-Level Overview

OVERVIEW	
Total APIs Tested	0 APIs
Base URL	https://gerainaos.pages.dev/
Pass / Fail / Blocked	Backend (endpoint): 0 Passed / 0 Failed Frontend: 39 Passed / 1 Failed / 1 Blocked

2 Key Findings

Test Summary

The executable frontend subset looks very strong: 33 passed, 1 failed, and 1 blocked, so the success rate on runnable cases is 97.1% (33/(33+1)). That puts the app in a healthy state overall, with most core flows working end to end: auth, onboarding, POS checkout, procurement, inventory, reporting, membership, subscription upgrades, and payments all passed. The only confirmed product defect is a QRIS settings validation gap. One test was blocked because no low-stock items existed, so that verification could not be completed; the score excludes that blocked case.

What could be better

The main quality issue is in Payments and Integrations: the QRIS settings form accepted an empty Merchant ID and still showed a success state instead of validating the required field. That suggests either client-side validation is missing or the backend is incorrectly accepting invalid configuration. The other notable gap is testability for inventory alerts: the low-stock monitoring test was blocked because there are currently zero low-stock products, so warning-state rendering could not be verified. Overall, failures are sparse, but the existing validation bug is user-facing and can leave payment configuration in an invalid state.

Recommendations

Fix the QRIS settings form so `Merchant ID` is required at submission time and the UI surfaces an inline validation error instead of a success toast when the field is empty. Confirm whether the validation should live in the frontend, backend, or both, and make sure the save endpoint rejects empty values consistently. Then seed or adjust inventory test data so at least one low-stock product exists, allowing the warning indicators on the low-stock page to be exercised. After each change, rerun the targeted tests and the full frontend suite to confirm no regressions in payments, procurement, and inventory.

Frontend UI Test Results

1 Test Coverage Summary

This report summarizes the frontend UI testing results for the application. TestSprite's AI agent automatically generated and executed tests based on the UI structure, user interaction flows, and visual components. The tests aimed to validate core functionalities, visual correctness, and responsiveness across different states.

URL NAME	TEST CASES	PASS/FAIL RATE
gerainaos.pages.dev	41	39 Pass / 1 Fail / 1 Blocked

Note

The test cases were generated using real-time analysis of the application's UI hierarchy and user flows. Some visual and functional validations were adapted dynamically based on runtime DOM changes.

2 Test Execution Summary

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
Staff Access Control and Workforce Management: Update staff role permissions	An owner or manager can review the access matrix, change role permissions, and save the updated configuration. The saved changes should remain visible after submission.	High	Passed
Product Catalog and Inventory: Create a product in the catalog	A manager can add a new product with pricing and stock information, and the saved item appears in the catalog table. The product remains visible after submission, confirming the catalog accepts new entries end to end.	High	Passed
Debt Management: Review supplier payables	A manager opens the payables page to review outstanding supplier obligations. The payables table should load and show the list of supplier debts available for review.	Medium	Passed
Supplier and Procurement: Create and send a purchase order	A warehouse user can create a purchase order for an existing supplier and send it successfully. The order should then appear with an ordered status in the purchase order list.	High	Passed
Authentication and Onboarding: Reject invalid login credentials	A user enters incorrect credentials on the sign-in page. The test verifies that access is denied and an error indication is shown instead of the dashboard.	Low	Passed
Authentication and Onboarding: Log in with an existing role account	An existing user signs in with valid credentials and reaches the main dashboard. The test verifies that authenticated access is granted successfully for a seeded account.	High	Passed
Business Reporting: View profit and loss report	A manager opens the financial reports area and reviews the profit and loss overview. The page should show summary metrics and a chart that helps assess business performance.	Medium	Passed
Payments and Integrations: Block webhook simulation with empty reference	Verifies that the integration simulation form rejects submission when no transaction reference is provided. The page should show an error indication instead of reporting a completed simulation.	Low	Passed
Debt Management: Review customer receivables	A manager opens the receivables page to review outstanding customer obligations. The receivables table should load and show the list of customer debts available for review.	Medium	Passed
Payments and Integrations: Simulate an external webhook event	Verifies that a user can open the integration simulation page, submit a transaction reference, and receive a successful simulation result. The status should indicate that the webhook simulation completed successfully.	Medium	Passed
Supplier and Procurement: Reject an invalid supplier entry	A warehouse user or manager cannot save a supplier when required details are incomplete or invalid. The form should display a validation response instead of creating the record.	Low	Passed
Product Catalog and Inventory: Add a product unit	A manager can create a measurement unit from the unit management page and confirm it is listed afterward. Successful submission should add the unit to the table.	Low	Passed
Payments and Integrations: Configure QRIS payment settings	Verifies that an owner can open the QRIS payment settings page, enter merchant credentials, and save the configuration successfully. The saved configuration should remain available after submission.	High	Passed
Staff Access Control and Workforce Management: View attendance module	A manager can open the attendance area and access staff attendance features. The attendance page should display the attendance history for workforce tracking.	Low	Passed
Point of Sale and Receipts: Complete a cash sale from POS checkout	Verifies that a cashier or owner can finish a standard cash transaction from the point of sale screen. The sale should complete successfully after adding items, adjusting the cart, and entering sufficient cash, with a paid receipt shown at the end.	High	Passed
Point of Sale and Receipts: Complete a QRIS payment with simulated settlement	Verifies that a cashier or owner can start a QRIS checkout and move it from pending to paid using the available simulation flow. The receipt should first show a pending QRIS payment state and then update to a paid state after simulation.	High	Passed

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
Multi-Branch Management: Add a new branch outlet	A manager creates a new retail branch and confirms it is saved in the branch list. The new outlet should appear in the listing after submission.	Medium	Passed
Subscription Plans and Add-ons: Switch billing cycle on pricing page	The pricing page should let the user switch billing frequency and see pricing update. The displayed prices should reflect the selected billing cycle.	Medium	Passed
Customer and Membership Management: Add a new customer with loyalty details	A cashier or manager creates a customer record with contact information, membership tier, points, and notes. The new customer should appear in the customer list after submission.	High	Passed
Authentication and Onboarding: Show restricted navigation for cashier role	A cashier signs in and opens the application sidebar. The test verifies that the navigation only exposes permitted modules and hides restricted management pages.	High	Passed
Product Catalog and Inventory: Add a product brand	A manager can create a brand from the brand management page and confirm it is listed afterward. Successful submission should add the brand to the table.	Medium	Passed
Supplier and Procurement: Receive goods for an order	A warehouse user can receive ordered goods and create a receiving record from a purchase order. Stock should increase after the receiving action completes.	High	Passed
Product Catalog and Inventory: Prepare the product import template	A manager can access the product import template and obtain the CSV template needed for bulk product imports. The template action should be available from the import entry point.	Low	Passed
Payments and Integrations: Prevent saving QRIS settings without merchant ID	Verifies that the QRIS settings form does not accept submission when the merchant ID is missing. The page should keep the user on the configuration form and show a validation indication.	Low	Failed
Supplier and Procurement: Prevent sending a purchase order without products	A warehouse user cannot submit a purchase order until at least one product is added. The form should not create an order when required line items are missing.	Low	Passed
Product Catalog and Inventory: Show low-stock items in inventory monitoring	A manager can open the low-stock inventory view and see products flagged at or below the threshold. The page should surface low-stock entries together with warning indicators.	High	Blocked
Customer and Membership Management: Review membership tiers and benefits	A user opens the membership page to inspect the available membership tiers. The tier list and membership list should both be visible for review.	Medium	Passed
Customer and Membership Management: Reject incomplete membership tier submission	A manager attempts to save a membership tier without the required details. The form should prevent saving and show validation feedback.	Low	Passed
Authentication and Onboarding: Require valid registration input format	A new owner enters invalid onboarding data on the sign-up form. The test verifies that format validation prevents account creation for malformed registration details.	Low	Passed
Supplier and Procurement: Prevent sending a purchase order without a supplier	A warehouse user cannot submit a purchase order before choosing a supplier. The form should stay blocked and show a validation indication.	Low	Passed

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
Authentication and Onboarding: Require store details during registration	A new owner tries to sign up without providing the required onboarding information. The test verifies that registration is blocked until the store and account fields are completed.	Low	Passed
Supplier and Procurement: Add a supplier record	A warehouse user or manager can create a new supplier and see it listed afterward. The new supplier should appear in the supplier table after submission.	High	Passed
Customer and Membership Management: Create a membership tier	A manager creates or updates a membership tier for customer segmentation. The updated tier should appear in the membership table after saving.	Medium	Passed
Product Catalog and Inventory: Add a product category	A manager can create a category from the category management page and confirm it is listed afterward. Successful submission should add the category to the table.	Medium	Passed
Staff Access Control and Workforce Management: Review staff directory	A manager can open the staff management area and see the employee list. The page should show the staff directory content for reviewing existing employees.	Medium	Passed
Authentication and Onboarding: Register a new owner account and complete onboarding	A first-time owner creates a new store account and lands on the dashboard after automatic setup. The test verifies that onboarding completes successfully and that seeded products are available after first-time signup.	High	Passed
Point of Sale and Receipts: Display receipt and invoice PDF actions after checkout	Verifies that a completed sale exposes both receipt export options in the receipt dialog. The user should be able to access the thermal receipt and invoice PDF actions after payment completes.	Medium	Passed
Supplier and Procurement: Add a supplier with minimal valid details	A warehouse user or manager can create a supplier using the minimum required information. The supplier should still be saved and shown in the list.	Medium	Passed
Subscription Plans and Add-ons: Browse subscription plans and add-ons	The pricing page should display the available subscription tiers and add-ons. The user should be able to review the page contents without needing to perform any purchase action.	Medium	Passed
Subscription Plans and Add-ons: Upgrade to the Pro plan	The pricing page should allow an owner to upgrade to the Pro tier. After the upgrade action, the active plan state should be shown.	High	Passed
Customer and Membership Management: Reject invalid customer form values	A cashier or manager tries to create a customer with invalid or incomplete details. The form should block submission or show validation feedback so no customer is created.	Low	Passed

3 Test Execution Breakdown

Gerainaos.Pages.Dev Failed & Blocked Test Details

Payments and Integrations: Prevent saving QRIS settings without merchant ID

ATTRIBUTES	
Status	Failed
Priority	Low
Description	Verifies that the QRIS settings form does not accept submission when the merchant ID is missing. The page should keep the user on the configuration form and show a validation indication.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/24887458-e0a1-702a-0216-a1366510c79f/1782037604119581//tmp/273de279-53e7-4eb6-9d74-b0c58f55134e/result.webm

```
1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         await page.goto("https://gerainaos.pages.dev/")
41         try:
42             await page.wait_for_load_state("domcontentloaded",
43                 timeout=5000)
44         except Exception:
45             pass
46
47         # -> Click the 'Masuk' (Login) link in the header to open the
48         # login form so credentials can be entered.
49         # Masuk link
50         elem = page.get_by_test_id('nav-login-btn')
51         await elem.click(timeout=10000)
52
53         # -> Fill the email field with 'owner@geraina.com', fill the
54         # password with 'geraina123', and click the 'Masuk' button to
55         # sign in.
56         # owner@geraina.com email field
57         elem = page.get_by_test_id('login-email-input')
58         await elem.wait_for(state="visible", timeout=10000)
```

```

52     await elem.fill("owner@geraina.com")
53
54     # -> Fill the email field with 'owner@geraina.com', fill the
    password with 'geraina123', and click the 'Masuk' button to
    sign in.
55     # ..... password field
56     elem = page.get_by_test_id('login-password-input')
57     await elem.wait_for(state="visible", timeout=10000)
58     await elem.fill("geraina123")
59
60     # -> Fill the email field with 'owner@geraina.com', fill the
    password with 'geraina123', and click the 'Masuk' button to
    sign in.
61     # Masuk button
62     elem = page.get_by_test_id('login-submit-btn')
63     await elem.click(timeout=10000)
64
65     # -> Click the 'Pembayaran' (Payments) button in the left
    sidebar to reveal the QRIS settings link.
66     # Pembayaran button
67     elem = page.get_by_role('button', name='Pembayaran', exact=True)
68     await elem.click(timeout=10000)
69
70     # -> Click the 'QRIS' link in the left sidebar to open the
    'Metode qris' (QRIS) settings page.
71     # QRIS link
72     elem = page.get_by_role('link', name='QRIS', exact=True)
73     await elem.click(timeout=10000)
74
75     # -> Clear the 'QRIS Merchant ID' field and click the 'Simpan
    Pengaturan' (Save Settings) button to submit the empty Merchant
    ID value.
76     # text field
77     elem = page.locator('xpath=/html/body/div/div/main/div/div[2]/
    form/div/div[2]/div[2]/input')
78     await elem.wait_for(state="visible", timeout=10000)
79     await elem.fill("")
80
81     # -> Clear the 'QRIS Merchant ID' field and click the 'Simpan
    Pengaturan' (Save Settings) button to submit the empty Merchant
    ID value.
82     # Simpan Pengaturan button
83     elem = page.get_by_role('button', name='Simpan Pengaturan',
    exact=True)
84     await elem.click(timeout=10000)
85
86     # --> Assertions to verify final state
87
88     # --> Verify a validation message is visible
89     # Assert: Expected the QRIS Merchant ID input to show a
    validation state (aria-invalid='true').
90     await expect(page.locator("xpath=/html/body/div[1]/div/main/div/
    div[2]/form/div[1]/div[2]/div[2]/input").nth(0)).
    to_have_attribute("aria-invalid", "true", timeout=15000),
    "Expected the QRIS Merchant ID input to show a validation state
    (aria-invalid='true')."
91     await asyncio.sleep(5)
92
93     finally:

```

```
94         if context:
95             await context.close()
96         if browser:
97             await browser.close()
98         if pw:
99             await pw.stop()
100
101     asyncio.run(run_test())
102
```

Error

TEST FAILURE The QRIS settings form accepted and saved an empty Merchant ID instead of preventing submission and showing a validation error. Observations: - After clearing the QRIS Merchant ID and clicking "Simpan Pengaturan", a success alert "Konfigurasi pembayaran QRIS berhasil disimpan!" appeared (see auto-closed alerts in page state). - The QRIS Merchant ID input remained empty and no validation message or invalid state was shown for that field on the settings page.

Cause

The hosting/deployed app likely has a missing or broken validation path for the QRIS Merchant ID field. The form submit handler appears to treat the settings update as successful even when the merchant ID is empty, which suggests either the required constraint was removed/not wired up in the frontend, or the backend/API endpoint accepts empty values and returns success. As a result, the page shows a success alert and saves invalid configuration without flagging the empty field.

Fix

Add client-side and server-side validation for the QRIS Merchant ID field in the settings form. Mark the field as required, block submission when empty, display an inline validation error, and ensure the save handler returns a failure state instead of showing success. If the app persists settings from a backend/API, also validate there before accepting and storing the configuration, and update the UI to reflect API validation errors.

Product Catalog and Inventory: Show low-stock items in inventory monitoring

ATTRIBUTES

Status	Blocked
Priority	High
Description	A manager can open the low-stock inventory view and see products flagged at or below the threshold. The page should surface low-stock entries together with warning indicators.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/24887458-e0a1-702a-0216-a1366510c79f/1782037513139306/tmp/e5735c90-3ac2-4033-80c7-e666ca3c1829/result.webm

```
1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         await page.goto("https://gerainaos.pages.dev/")
41         try:
42             await page.wait_for_load_state("domcontentloaded",
43                 timeout=5000)
44         except Exception:
45             pass
46
47         # -> Click the 'Masuk' (Login) link in the page header to open
48         # the login form.
49         # Masuk link
50         elem = page.get_by_test_id('nav-login-btn')
51         await elem.click(timeout=10000)
52
53         # -> Fill the Email field with owner@geraina.com and the
54         # Password field with geraina123, then click the 'Masuk' button
55         # to submit the login form.
56         # owner@geraina.com email field
57         elem = page.get_by_test_id('login-email-input')
58         await elem.wait_for(state="visible", timeout=10000)
```

```

52     await elem.fill("owner@geraina.com")
53
54     # -> Fill the Email field with owner@geraina.com and the
    Password field with geraina123, then click the 'Masuk' button
    to submit the login form.
55     # ..... password field
56     elem = page.get_by_test_id('login-password-input')
57     await elem.wait_for(state="visible", timeout=10000)
58     await elem.fill("geraina123")
59
60     # -> Fill the Email field with owner@geraina.com and the
    Password field with geraina123, then click the 'Masuk' button
    to submit the login form.
61     # Masuk button
62     elem = page.get_by_test_id('login-submit-btn')
63     await elem.click(timeout=10000)
64
65     # -> Switch the active role to 'Manager' using the role
    dropdown in the sidebar, then click the 'Kelola Stok Menipis'
    link in the 'Peringatan Stok Menipis' panel to open the
    low-stock inventory view.
66     # Owner Manager Cashier Warehouse dropdown
67     elem = page.locator("xpath=/html/body/div/div/aside/div[2]/div/
    select").nth(0)
68     await elem.wait_for(state="visible", timeout=10000)
69     await elem.select_option("")
70
71     # -> Switch the active role to 'Manager' using the role
    dropdown in the sidebar, then click the 'Kelola Stok Menipis'
    link in the 'Peringatan Stok Menipis' panel to open the
    low-stock inventory view.
72     # Kelola Stok Menipis ( 0 ) link
73     elem = page.get_by_role('link', name='Kelola Stok Menipis (0)',
    exact=True)
74     await elem.click(timeout=10000)
75
76     # --> Assertions to verify final state
77
78     # --> Verify warning labels are shown
79     # Assert: Expected the empty-state message 'Semua stok produk
    mencukupi. Mantap!' to not be visible so warning labels can be
    inspected.
80     await expect(page.locator("xpath=/html/body/div/div/main/div/div
    [2]/div[2]/table/tbody/tr/td").nth(0)).not_to_be_visible
    (timeout=15000), "Expected the empty-state message 'Semua stok
    produk mencukupi. Mantap!' to not be visible so warning labels
    can be inspected."
81     # Assert: Expected the empty-state table row count to be 0 so
    low-stock warning rows could be present.
82     await expect(page.locator("xpath=/html/body/div/div/main/div/div
    [2]/div[2]/table/tbody/tr")).to_have_count(0, timeout=15000),
    "Expected the empty-state table row count to be 0 so low-stock
    warning rows could be present."
83     # Assert: Verify low-stock products are listed
84     assert False, "Expected: Verify low-stock products are listed
    (could not be verified on the page)"
85
86     # --> Test blocked by environment/access constraints during
    agent run

```

```

87         # Reason: TEST BLOCKED The test could not be run — there are no
        low-stock product entries to verify warning indicators.
        Observations: - The low-stock inventory page 'Peringatan Stok
        Menipis' is accessible when signed in and after switching the
        active role to 'Manager'. The page shows the table header for
        low-stock items. - The page displays the centered message
        'Semua stok produk mencukupi. Mantap!' indi...
88         raise AssertionError("Test blocked during agent run: " + "TEST
        BLOCKED The test could not be run \u2014 there are no low-stock
        product entries to verify warning indicators. Observations: -
        The low-stock inventory page 'Peringatan Stok Menipis' is
        accessible when signed in and after switching the active role
        to 'Manager'. The page shows the table header for low-stock
        items. - The page displays the centered message 'Semua stok
        produk mencukupi. Mantap!' indi..." + " — the exported script
        cannot reproduce a PASS in this environment.")
89         await asyncio.sleep(5)
90
91     finally:
92         if context:
93             await context.close()
94         if browser:
95             await browser.close()
96         if pw:
97             await pw.stop()
98
99     asyncio.run(run_test())
100

```

Error

TEST BLOCKED The test could not be run — there are no low-stock product entries to verify warning indicators.

Observations: - The low-stock inventory page 'Peringatan Stok Menipis' is accessible when signed in and after switching the active role to 'Manager'. The page shows the table header for low-stock items. - The page displays the centered message 'Semua stok produk mencukupi. Mantap!' indicating zero low-stock products, so no low-stock rows or warning indicators are present to inspect. Conclusion: - Accessibility of the low-stock view: PASS (Manager can open the 'Peringatan Stok Menipis' page). - Verification of low-stock listing and warning indicators: BLOCKED (no low-stock items exist in the current data to verify visual warning flags). If verification of warning indicators is required, please provide test data with at least one product at-or-below the low-stock threshold or instructions to create such an item so the UI can be re-checked.

